arXiv:2411.12240v2 [cs.CL] 26 Nov 2024

EVALUATING TOKENIZER PERFORMANCE OF LARGE LANGUAGE MODELS ACROSS OFFICIAL INDIAN LANGUAGES

A PREPRINT

Sagar Tamang*
Department of IT
The Assam Kaziranga University
Jorhat, India
cs22bcagn033@kazirangauniversity.in

Dr. Dibya Jyoti Bora
Department of IT
The Assam Kaziranga University
Jorhat, India
dibyajyotibora@kazirangauniversity.in

November 27, 2024

ABSTRACT

Large Language Models (LLMs) based on transformer architectures have revolutionized a variety of domains, with tokenization playing a pivotal role in their pre-processing and fine-tuning stages. In multilingual models, particularly those tailored for Indic languages, effective tokenization is crucial for optimizing performance. This paper presents a comprehensive evaluation of tokenizers used by 12 LLMs across all 22 official languages of India, with a focus on comparing the efficiency of their tokenization processes. We employed the Normalized Sequence Length (NSL) as a key metric in our analysis. Our findings reveal that the SUTRA tokenizer outperforms all other models, including several Indic-specific models, excelling in 14 languages. Notable insights include the SUTRA tokenizer's superior handling of Indic languages, GPT-4o's advancement over its predecessor GPT-4 in processing Indian languages, and the limited performance of Project Indus in certain languages. This study underscores the critical importance of developing targeted tokenization strategies for multilingual and Indic-centric models, laying the groundwork for future improvements in tokenizer design to enhance linguistic coverage and model efficiency.

Keywords tokenizer · LLM · tokens · GPT · SUTRA · indic languages.

1 Introduction

1.1 Background

In an ever-evolving landscape of Artificial Intelligence (AI), transformers-based generative Large Language Models (LLMs) are transforming an increasing number of fields with an ever-increasing number of applications in finance, medicine, education, and many more [1, 2]. Tokenization is an important step for LLMs, especially in pre-processing and fine-tuning stages [3].

Most of the LLMs use either of two types of tokenization algorithms, namely WordPiece and Byte Pair Encoding (BPE). For example, OpenaAi's GPT-4o model and META's Llama 3, both employ a modified BPE tokenizer [4]. WordPiece was developed for models like BERT employing a greedy approach. It starts with the longest substring that matches a token in its vocabulary, allowing it to handle out-of-vocabulary words effectively by breaking them down into known subword units [5, 6]. BPE works by iteratively merging the most frequently occurring pairs of characters or subwords in a corpus to create a complete vocabulary [7, 8].

Agnostic of the tokenization algorithms used, many techniques have been developed to compare tokenizers of LLMs. Subword fertility is one such technique which measures the average number of tokens used per word [9]. Normalized Sequence Length (NSL) is another such metric used to evaluate the efficiency of tokenizers [12].

*Correspondance can be addressed to *cs22bcagn033@kazirangauniversity.in*

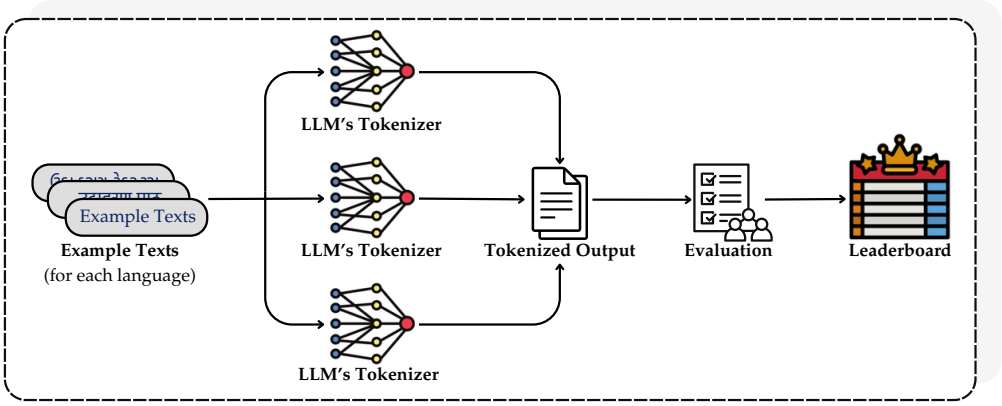

Figure 1: Evaluation pipeline: (1) We collect example texts for all 22 languages. (2) We send the example texts to the LLMs' tokenizer. (3) Evaluate the tokenized outputs. (4) We construct leaderboards using our evaluation.

An effective tokenizer is essential for enabling a model to learn a language efficiently. Improved tokenizer performance offers several benefits, including faster token generation and reduced computational resource requirements, enhancing both efficiency and cost-effectiveness [13].

1.2 Tokenizers in Multilingual & Indic Models

Many multilingual models, including OpenAI's ChatGPT [14], Google's Gemini [16], Meta's Llama [17], and TWO AI's SUTRA [15], are designed to deliver coherent performance across a wide range of global languages, including Indian languages. Achieving this requires tokenizers in these LLMs to handle diverse languages efficiently. This study evaluates the performance of tokenizers in these multilingual models and Indic-specific models across all official languages of India.

The rest of the paper is organized in the following way: Section 2 highlights related works, Section 3 describes the methodology of this study, Section 4 is where the results are showcased and Section 5 is for discussions with future outlook.

2 Literature Review

2.1 Large Language Models and Tokenization

Ever since the release of transformer-based architecture, Large Language Models (LLMs) have demonstrated remarkable capabilities in Natual Language Processing (NLP) tasks and beyond [19]. However, some recent studies [18] have also proposed promising non-transformer LLMs. LLMs today exhibit a plethora of applications not limited to NLP tasks, but can also perform general tasks performing multi-step reasoning. Thus, LLMs are becoming the basic building block for developing general-purpose AI agents or Artificial General Intelligence (AGI) [18].

Tokenization refers to the process of converting a sequence of texts into smaller parts, known as tokens. In order to increase the coverage of dictionaries and also to deal with words that were unseen in training data, LLMs use sub-words based tokenizers like BPE or Wordpiece [4, 8, 7, 18].

2.2 Indic-Specific Language Models

S. Bhat et al. evaluated the capabilities of generative models like ChatGPT, mT0, and BLOOMZ in generating Indic languages, and the findings revealed that these models have limited capabilities in generating text in Indic languages in a zero-shot setting. While they performed better on manual quality evaluations for English, their performance in Indic languages highlighted the need for further development and training specifically tailored to these languages [22]. Several studies have introduced benchmarks to evaluate the performance of LLMs across Indic languages, and their results emphasize the necessity for more focused research and development efforts in LLMs to handle the linguistic diversity of India [23, 24, 25].

One notable advancement is SUTRA (Scalable Multilingual Language Model Architecture), introduced by A. Bendale et al. SUTRA supports over 50 languages, including Indic ones, by decoupling conceptual understanding from language-

specific processing for scalable multilingual learning. It uses a Mixture of Experts framework, enhancing computational efficiency and responsiveness. Evaluations show that SUTRA outperforms models like GPT-3.5 and Llama2 by 20-30% on MMLU benchmarks. The model provides hallucination-free, factual, and up-to-date responses, with the potential to democratize AI access, especially in non-English regions [15, 29].

2.3 Comparative Evaluation of Tokenizers

To compare the tokenizers, several metrics and methodologies have been introduced. Subword fertility is one such metric that measures the average number of tokens generated per word [12]. An ideal tokenizer in this case would have a fertility of 1.0, indicating that most words are represented as single tokens, while a higher fertility score signals that many words are split into multiple tokens [10]. Another metric that is employed is the proportion of continued words [10] which indicates the percentage of words that are split into multiple tokens. 0 is an ideal value for the proportion of continued words. A lower proportion indicates better performance, as it means fewer words are fragmented [10]. Another metric that is used to evaluate and compare tokenizers is the Normalized Sequence Length (NSL). NSL measures the average length of tokenized sequences produced by a tokenizer relative to a baseline tokenizer [12].

Existing tokenizer studies are performed at a smaller scale than what is typical for modern LLMs or focus on multilingual tasks [12]. Some studies have tried to evaluate the performance of tokenizers but only in some select languages like Turkish [21], Arabic [26], or focusing on a few Indian languages [3, 20].

2.4 Gaps in Existing Research

Thus, we can observe that there exists a research gap to evaluate the tokenizers in other Indian languages as well. Hence, in this study, we are conducting an overall evaluation of the tokenizers of LLMs in all 22 official languages of India as recognized by the eighth schedule of the Indian constitution [27].

3 Methodology

Our evaluation step is summarized in Figure 1.

3.1 Example Texts

Example Text in Assamese

"জীৱনৰ পৰিসৰে মোহিত হোৱাটো বাঞ্ছনীয়।"
Translation: "It is desirable to be captivated by the expanse of life."
Source: "Miri Jiyori" by Rajanikanta Bordoloi

Figure 2: Assamese text used for evaluating tokenizer performance.

We compiled example texts in all 22 languages to evaluate the performance of the tokenizers. Each language's text was selected in its primary writing script to ensure an authentic assessment of the tokenizer's capability to process native scripts accurately. The curated example texts represent diverse linguistic structures and scripts, enabling a comprehensive analysis of tokenization performance.

All the example texts used during our study can be found in the Appendix A.2 or Figure 26 and 27. One such example text can be found in Figure 2.

3.2 Models

We chose 12 models including proprietary multilingual models, as well as Open-weights multilingual and Indic language models for our study. The list of models can be found in Table 1.

While we acknowledge that some models are not specifically designed for all Indian languages—such as *MahaMarathi*, which is tailored for Marathi, and *MBZUAI's Nanda*, which is optimized for Hindi and English—we have included them in this study for the sake of comprehensive evaluation.

Models	Languages Tested	Availability
GPT-4o	All	Proprietary
GPT-4	All	Proprietary
TWO/sutra-mlt256-v2	All	Proprietary
microsoft/Phi-3.5-MoE-instruct	All	Open-weights
meta-llama/Llama-3.1-405B-FP8	All	Open-weights
ai4bharat/Airavata	All	Open-weights
CohereForAI/aya-23-35B	All	Open-weights
MBZUAI/Llama-3-Nanda-10B-Chat	All	Open-weights
nickmalhotra/ProjectIndus	All	Open-weights
sarvamai/OpenHathi-7B-Hi-v0.1-Base	All	Open-weights
Telugu-LLM-Labs/Indic-gemma-7b-finetuned-sft-Navarasa-2.0	All	Open-weights
marathi-llm/MahaMarathi-7B-v24.01-Base	All	Open-weights

Table 1: List of tokenizers tested. “All" refers to all 22 official languages of India as recognized by the Eighth Schedule of the Indian Constitution. The official languages include Assamese, Bengali, Bodo, Dogri, Gujarati, Hindi, Kannada, Kashmiri, Konkani, Maithili, Malayalam, Manipuri, Marathi, Nepali, Odia, Punjabi, Sanskrit, Santali, Sindhi, Tamil, Telugu, Urdu.

3.3 Evaluation Metric

For our work is extending the previous works by [3], we have chosen to go with the NSL metric. Formally the NSL is defined by [12] $c_{\lambda\beta}$ as the ratio between the length of an encoded sequence from a tokenizer T_λ and a tokenizer T_β. For N examples taken from a dataset D:

$$c_{\lambda\beta} = \frac{\sum_{i=1}^{N} \text{length}(T_\lambda(D_i))}{\sum_{i=1}^{N} \text{length}(T_\beta(D_i))}$$

4 Results

Average NSL Values Table 2 presents the average NSL values for all tokenizers across the 22 languages, calculated using the examples provided in Appendix A.2. The scores are reported to four decimal places for precision. The **bold text** indicates the lowest value or the best performance among all the other tokenizers. It can be observed that SUTRA tokenizer manages to excel among all the other tokenizers including ChatGPT’s 4-o or other Indic models.

Figure 3 illustrates the number of languages in which each tokenizer achieved the highest NSL score. For instance, TWO AI’s SUTRA outperformed all other tokenizers in 14 languages, while MBZUAI’s Nanda excelled in 6, and OpenAI’s GPT-4o in 5. Other notable performers include indic models like Tech Mahindra’s Project Indus with 4, Sarvam AI’s OpenHathi and MahaMarathi with 2 each, and Indic Gemma, Microsoft Phi, and Airavata with 1 each.

Number of Tokens Appendix A.1 provides individual row bar charts for each language, offering a detailed breakdown of the number of tokens generated in each language. Lower token counts indicate better outcomes.

5 Discussion

In this study, we evaluated the tokenizers from 12 LLMs in all 22 official languages of India and we found that the SUTRA tokenizer performed the best among all others, outperforming the 2nd best tokenizer by a large margin. This showcases the multilingual strength of the SUTRA tokenizer to handle the Indic languages.

Microsoft’s Phi-3.5-MoE-instruct and Google’s Indic Gemma Though both the models were developed for Indic languages, they did not perform up to the level securing best performances only in one language out of 22 languages.

Observation between GPT-4 and GPT-4o Another interesting observation is that GPT-4, the predecessor of GPT-4o, did not manage to secure the best tokenizer value in any of the 22 languages, a stark contrast to GPT-4o. Perhaps, this highlights that an important difference between the two models is that the newer GPT-4o is well adept at Indian languages, increasing the multi-lingual capability.

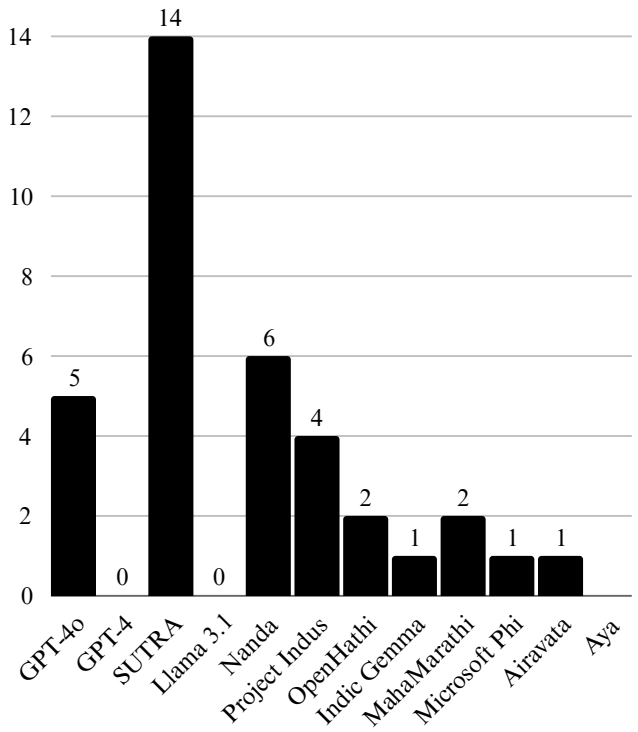

Figure 3: Number of Best Performances Achieved by Each Tokenizer Across 22 Languages.

Languages	GPT-4o	GPT-4	SUTRA	Llama 3.1	Nanda	Project Indus	OpenHathi	Indic Gemma	MahaMarathi	Microsoft Phi	Airavata	Aya
Assamese	0.5429	1.4	**0.4571**	1.4	1.4	2.7714	1.5714	0.8286	1.3143	1.5428	1.5714	1.5142
Bengali	0.25	1.2307	**0.2115**	1.25	1.25	2.8076	1.3461	0.5769	1.0961	1.3269	1.3461	1.2307
Bodo	0.5675	1.0540	0.5405	0.5945	**0.4594**	0.4864	0.5405	0.5675	0.5405	1.2432	0.5405	0.7567
Dogri	0.5	1.0313	0.4688	0.5938	**0.3750**	0.4688	0.4063	0.4688	0.4063	1.0312	0.4062	0.7812
Gujarati	0.47545	1.6875	**0.4688**	1.7188	1.7188	2.75	2.6875	0.7188	1.0938	2.6562	2.6875	1.7187
Hindi	0.4091	1.0	0.4545	0.5909	0.3636	**0.3182**	0.4545	0.5455	0.3636	1.0454	0.4545	1.0454
Kannada	**0.44**	1.76	**0.44**	1.8	1.8	2.84	2.52	0.56	1.12	2.48	2.52	1.72
Kashmiri	0.6047	1.093	**0.5814**	0.8837	0.8837	1.8605	1.1628	0.5814	1.186	1.1395	1.1627	0.9069
Konkani	**0.4643**	1.1429	0.5357	0.6429	0.5	0.4643	0.6071	0.5357	1.1071	0.6071	0.6071	0.8214
Maithili	0.4211	1.0	0.6316	0.6316	**0.3684**	**0.3684**	0.5789	0.5789	1.1578	0.5789	0.5789	0.7368
Malayalam	**0.5**	1.75	**0.5**	1.8333	1.8333	3.0	1.3333	0.6667	1.25	1.3333	1.3333	1.8333
Manipuri	0.6471	1.2941	**0.5882**	1.3529	1.3529	2.8824	1.5882	0.7647	1.5882	1.5294	1.5882	1.2941
Marathi	0.4706	0.9412	0.5294	0.6471	**0.3529**	**0.3529**	0.4706	0.5882	1.0582	0.4706	0.4705	0.8235
Nepali	0.4091	0.9091	**0.3182**	0.6364	**0.3182**	0.3636	**0.3182**	0.4545	0.4090	1.1363	**0.3182**	0.8181
Odia	1.0	2.625	**0.625**	2.625	2.625	2.875	2.875	1.0625	2.875	2.8125	2.875	2.1875
Punjabi	0.6538	1.6923	**0.4615**	1.7308	1.7308	2.7692	2.3077	0.7692	2.3077	2.2692	2.3076	1.7307
Sanskrit	**0.5**	1.0833	0.6667	0.5833	**0.5**	**0.5**	**0.5**	0.6667	1.08333	**0.5**	**0.5**	0.75
Santali	2.7647	2.647	**0.4705**	2.7058	2.7058	2.8823	2.8823	1.0588	2.9411	2.8235	2.8823	2.7058
Sindhi	**0.4117**	0.9117	0.5	0.6176	0.6176	1.8529	1.0882	0.5588	1.1176	1.0588	1.0882	0.5882
Tamil	0.4411	1.3823	**0.3823**	1.4117	1.4117	2.7647	1.2352	0.5294	1.0882	1.2058	1.2352	1.2058
Telugu	0.375	1.75	**0.2916**	1.7916	1.7916	2.8333	2.6666	0.625	1.125	2.625	2.6666	1.7083
Urdu	0.3928	0.7857	**0.3571**	0.5357	1.8928	1.8928	1.1071	0.4285	1.1071	1.0714	1.1071	0.5357

Table 2: Average NSL Values Across Models for 22 Languages (lower is better). The bold values indicate the best-performing tokenizer for each language.

Observation of Tech Mahindra's Project Indus Both SUTRA and GPT-4o tokenizers manage to get a consistently low average NSL value (below 1.0) for all the languages but the Project Indus' tokenizer seems to be getting the same for only a few languages like (1) Bodo, (2) Dogri, (3) Hindi, (4) Konkani, (5) Maithili, (6) Marathi, (7) Nepali, and (8) Sanskrit. This is probably because all these 8 languages follow the same Devanagari script of writing, which the model's tokenizer was probably trained on. But for the rest of the languages, the tokenizer seems to be struggling, getting an average NSL score of above 1 (the higher the worse).

Number of Tokens According to the results and appendix A.1, tokenizers like SUTRA generate fewer tokens across the 22 Indian languages compared to others. Lower token counts suggest that the tokenizer is more efficient in

processing the input text without excessive fragmentation. This is a crucial factor in improving the overall performance and computational efficiency of LLMs, particularly for large-scale applications.

Significance of tokenization in LLMs Tokenization plays a vital role in LLMs by breaking down text into smaller units (tokens) that the model can process efficiently. A well-designed tokenizer enables the model to handle complex language structures, out-of-vocabulary words, and multi-language contexts effectively. It enhances the model's ability to understand and generate language with greater accuracy. Additionally, a good tokenizer leads to reduced computational costs and resource requirements by optimizing token generation. This results in faster training times, lower resource consumption, and overall improved performance, allowing the model to process diverse languages more effectively.

Real-World Applications and Future Directions The insights from this study have important implications for the development of multilingual models across Indian languages. Future research could focus on enhancing tokenizers to better handle languages with complex scripts or languages with a high degree of dialectical variation, improving model performance for both high-resource and low-resource languages.

6 Acknowledgement

We would like to thank *the Assam Kaziranga University* for assisting us in conducting this research.

References

[1] F. Chiarello, V. Giordano, I. Spada, S. Barandoni, and G. Fantoni, "Future applications of generative large language models: A data-driven case study on ChatGPT," *Technovation*, vol. 133, p. 103002, 2024. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S016649722400052X`. [Accessed: Nov. 12, 2024].

[2] Y. Nie, Y. Kong, X. Dong, J. M. Mulvey, H. V. Poor, Q. Wen, and S. Zohren, "A Survey of Large Language Models for Financial Applications: Progress, Prospects and Challenges," *arXiv preprint arXiv:2406.11903*, 2024. [Online]. Available: `https://arxiv.org/abs/2406.11903`. [Accessed: Nov. 12, 2024].

[3] S. Tamang and D. J. Bora, "Performance Evaluation of Tokenizers in Large Language Models for the Assamese Language," arXiv preprint arXiv:2410.03718, 2024. [Online]. Available: `https://arxiv.org/abs/2410.03718`

[4] J. Yang, Z. Wang, Y. Lin, and Z. Zhao, "Large Language Model Tokenizer Bias: A Case Study and Solution on GPT-4o," *arXiv preprint arXiv:2406.11214*, 2024. [Online]. Available: `https://arxiv.org/abs/2406.11214`. [Accessed: Nov. 13, 2024].

[5] X. Song, A. Salcianu, Y. Song, D. Dopson, and D. Zhou, "Fast WordPiece Tokenization," 2021. [Online]. Available: `https://arxiv.org/abs/2012.15524`.

[6] O. Ogundepo, X. Zhang, and J. Lin, "Better Than Whitespace: Information Retrieval for Languages without Custom Tokenizers," 2022. [Online]. Available: `https://arxiv.org/abs/2210.05481`.

[7] L. Kozma and J. Voderholzer, "Theoretical Analysis of Byte-Pair Encoding," 2024. [Online]. Available: `https://arxiv.org/abs/2411.08671`.

[8] V. Zouhar, C. Meister, J. L. Gastaldi, L. Du, T. Vieira, M. Sachan, and R. Cotterell, "A Formal Perspective on Byte-Pair Encoding," 2024. [Online]. Available: `https://arxiv.org/abs/2306.16837`.

[9] H. Singh, N. Gupta, S. Bharadwaj, D. Tewari, and P. Talukdar, "IndicGenBench: A Multilingual Benchmark to Evaluate Generation Capabilities of LLMs on Indic Languages," *arXiv preprint arXiv:2404.16816*, 2024. [Online]. Available: `https://arxiv.org/abs/2404.16816`.

[10] Occiglot, "EU Tokenizer Performance," [Online]. Available: `https://occiglot.eu/posts/eu_tokenizer_perfomance/`. Accessed: Nov. 17, 2024.

[11] Occiglot, "Tokenizer performance on EU languages," Occiglot Blog, Sep. 26, 2023. [Online]. Available: `https://occiglot.eu/posts/eu_tokenizer_perfomance/`.

[12] G. Dagan, G. Synnaeve, and B. Rozière, "Getting the most out of your tokenizer for pre-training and domain adaptation," *arXiv preprint arXiv:2402.01035*, 2024. [Online]. Available: `https://arxiv.org/abs/2402.01035`.

[13] Microsoft, "Exploring the New Frontier of AI: OpenAI's GPT-4-O for Indic Languages," Azure AI Blog, Oct. 30, 2024. [Online]. Available: https://techcommunity.microsoft.com/blog/azure-ai-services-blog/exploring-the-new-frontier-of-ai-openais-gpt-4-o-for-indic-languages/4142383.

[14] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, *et al.*, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2024. [Online]. Available: https://arxiv.org/abs/2303.08774.

[15] A. Bendale, M. Sapienza, S. Ripplinger, S. Gibbs, J. Lee, and P. Mistry, "SUTRA: Scalable Multilingual Language Model Architecture," arXiv preprint arXiv:2405.06694, 2024. [Online]. Available: https://arxiv.org/abs/2405.06694.

[16] Gemini Team et al., *Gemini: A Family of Highly Capable Multimodal Models*, 2024. Available at: https://arxiv.org/abs/2401.12345.

[17] Dubey, Abhimanyu, et al. *The Llama 3 Herd of Models*. arXiv preprint, 2024. Available at: https://arxiv.org/abs/2407.21783.

[18] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao, "Large Language Models: A Survey," arXiv preprint arXiv:2402.06196, 2024. [Online]. Available: https://arxiv.org/abs/2402.06196

[19] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian, "A Comprehensive Overview of Large Language Models," arXiv preprint arXiv:2307.06435, 2024. [Online]. Available: https://arxiv.org/abs/2307.06435

[20] AdaSci, "Multilingual Tokenization Efficiency in Large Language Models: A Study on Indian Languages," [Online]. Available: https://adasci.org/multilingual-tokenization-efficiency-in-large-language-models-a-study-on-indian-languages/

[21] Cagri Toraman, Eyup Halit Yilmaz, Furkan Şahinüç, and Oguzhan Ozcelik, "Impact of Tokenization on Language Models: An Analysis for Turkish," ACM Transactions on Asian and Low-Resource Language Information Processing (TALLIP), vol. 22, no. 4, article 116, pp. 1–21, Mar. 2023. [Online]. Available: https://doi.org/10.1145/3578707

[22] Bhat, Savita, Vasudeva Varma, and Niranjan Pedanekar. "Generative Models For Indic Languages: Evaluating Content Generation Capabilities." In *Proceedings of the 14th International Conference on Recent Advances in Natural Language Processing*, edited by Ruslan Mitkov and Galia Angelova, 187–195. Varna, Bulgaria: INCOMA Ltd., September 2023. https://aclanthology.org/2023.ranlp-1.21.

[23] Singh, Harman, Nitish Gupta, Shikhar Bharadwaj, Dinesh Tewari, and Partha Talukdar. "IndicGenBench: A Multilingual Benchmark to Evaluate Generation Capabilities of LLMs on Indic Languages." In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, edited by Lun-Wei Ku, Andre Martins, and Vivek Srikumar, 11047–11073. Bangkok, Thailand: Association for Computational Linguistics, August 2024. https://aclanthology.org/2024.acl-long.595, DOI: 10.18653/v1/2024.acl-long.595.

[24] Singh, Abhishek Kumar, Rudra Murthy, Vishwajeet Kumar, Jaydeep Sen, and Ganesh Ramakrishnan. "Indic QA Benchmark: A Multilingual Benchmark to Evaluate Question Answering Capability of LLMs for Indic Languages." arXiv preprint, 2024. https://arxiv.org/abs/2407.13522.

[25] Kumar, Aman, Himani Shrotriya, Prachi Sahu, Amogh Mishra, Raj Dabre, Ratish Puduppully, Anoop Kunchukuttan, Mitesh M. Khapra, and Pratyush Kumar. "IndicNLG Benchmark: Multilingual Datasets for Diverse NLG Tasks in Indic Languages." In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, edited by Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, 5363–5394. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, December 2022. https://aclanthology.org/2022.emnlp-main.360, DOI: 10.18653/v1/2022.emnlp-main.360.

[26] Alyafeai, Zaid, Maged S. Al-shaibani, Mustafa Ghaleb, and Irfan Ahmad. "Evaluating Various Tokenizers for Arabic Text Classification." arXiv preprint, 2021. https://arxiv.org/abs/2106.07540.

[27] Government of India, "Eighth Schedule," [Online]. Available: https://www.mea.gov.in/Images/pdf1/S8.pdf. Accessed: Dec. 5, 2023.

[28] I. Watts, V. Gumma, A. Yadavalli, V. Seshadri, M. Swaminathan, and S. Sitaram, "PARIKSHA: A Large-Scale Investigation of Human-LLM Evaluator Agreement on Multilingual and Multi-Cultural Data," *arXiv preprint arXiv:2406.15053*, 2024. [Online]. Available: https://arxiv.org/abs/2406.15053

[29] DeepLearning.ai, *Startup TWO AI Launches SUTRA: A Multilingual Model for South Asian Markets*, https://www.deeplearning.ai/the-batch/startup-two-ai-launches-sutra-a-multilingual-model-for-south-asian-markets/, Accessed: 2024-11-18.

A Appendix

A.1 Bar Charts of Token Counts for Each Language

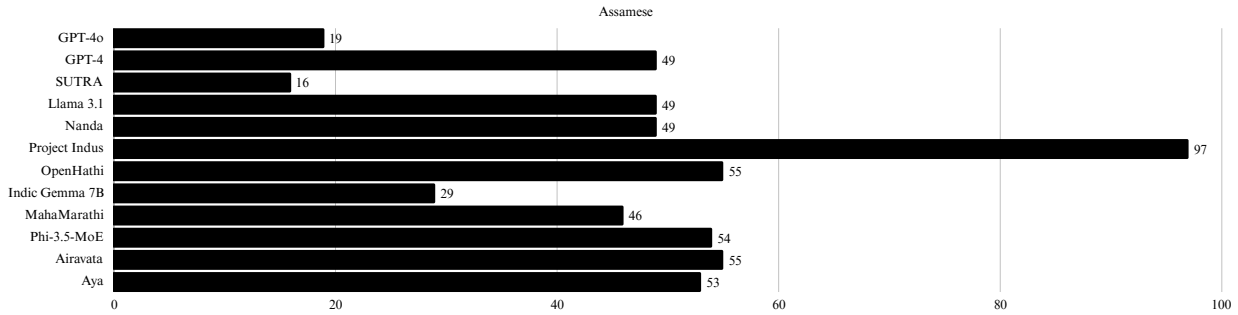

Figure 4: Number of tokens required for a single example text in Assamese. Lower values are better.

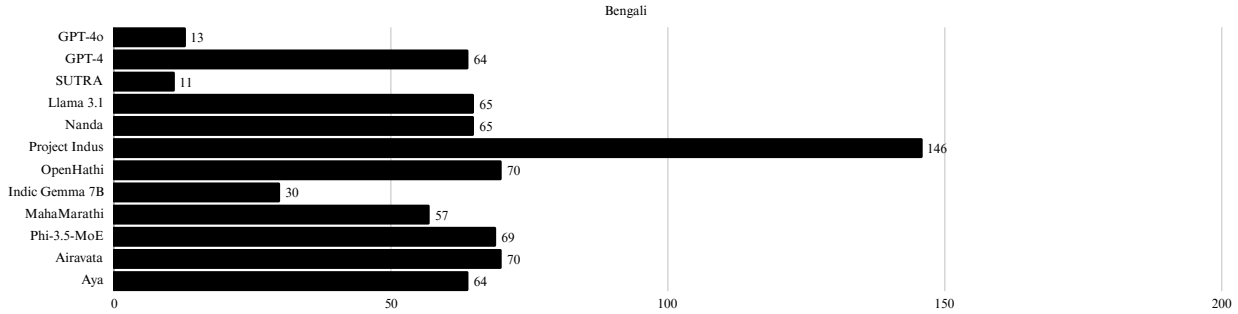

Figure 5: Number of tokens required for a single example text in Bengali. Lower values are better.

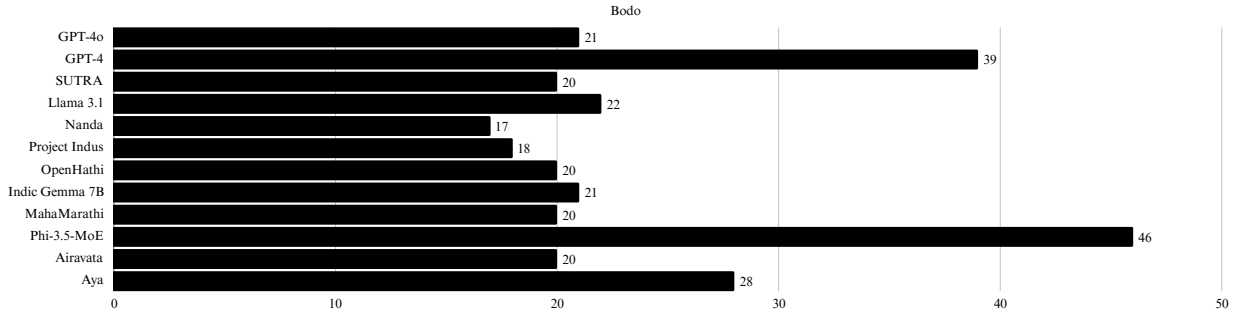

Figure 6: Number of tokens required for a single example text in Bodo. Lower values are better.

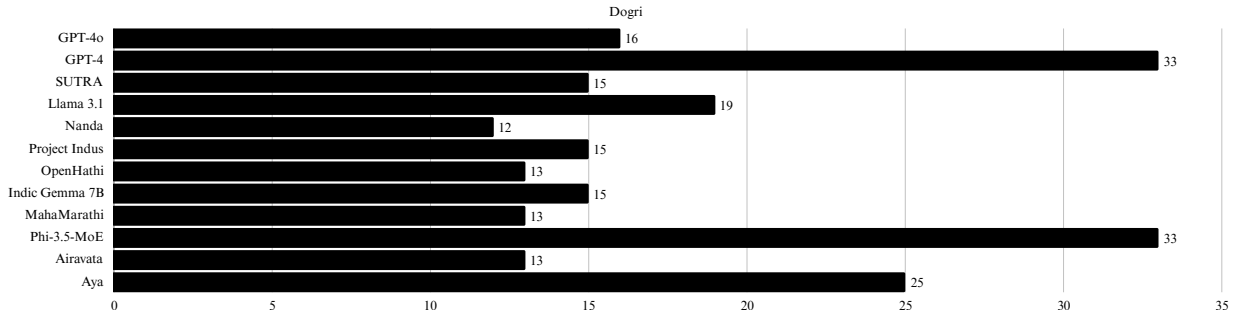

Figure 7: Number of tokens required for a single example text in Dogri. Lower values are better.

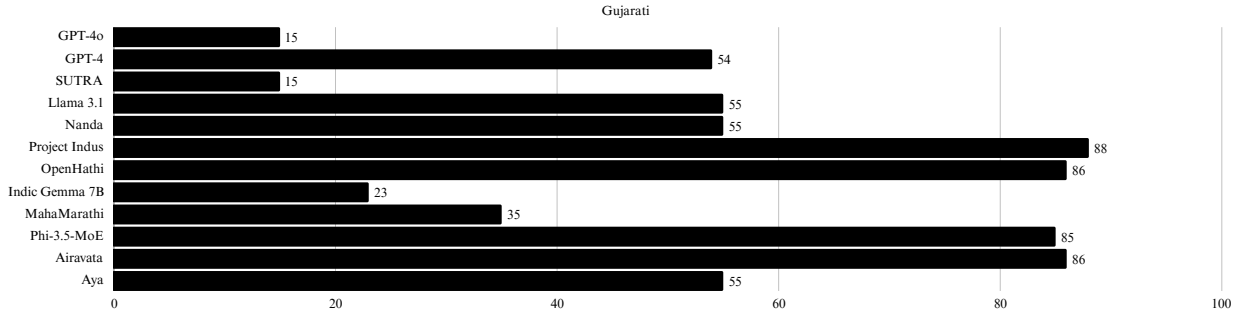

Figure 8: Number of tokens required for a single example text in Gujarati. Lower values are better.

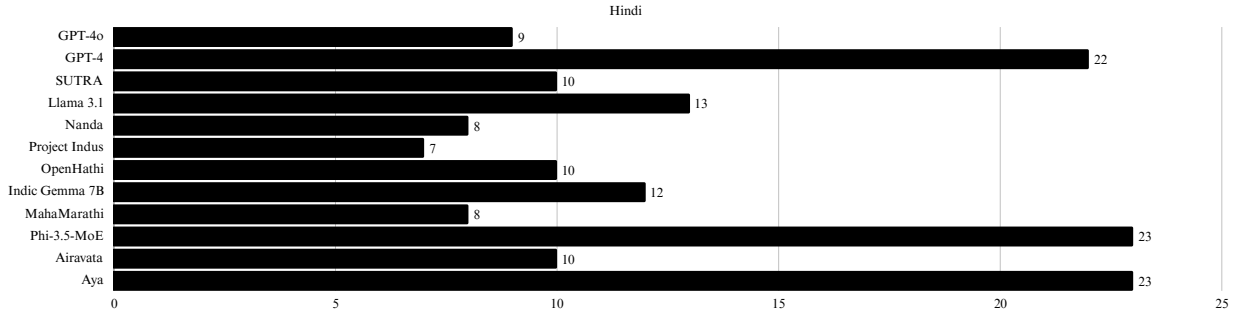

Figure 9: Number of tokens required for a single example text in Hindi. Lower values are better.

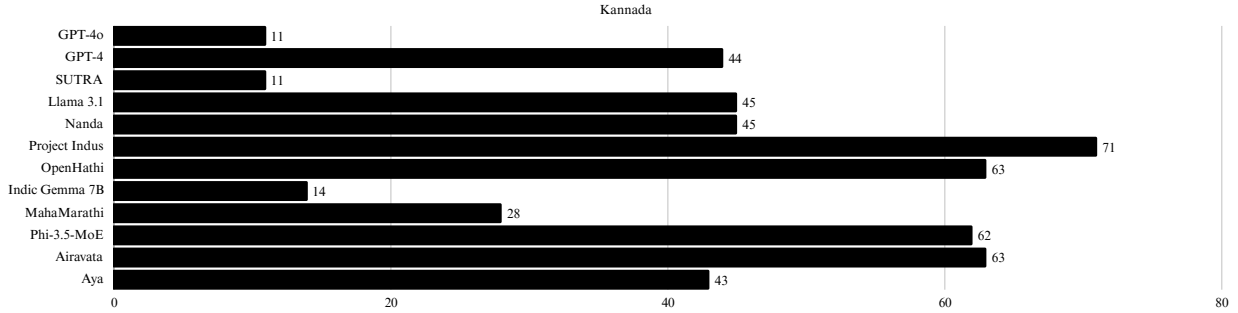

Figure 10: Number of tokens required for a single example text in Kannada. Lower values are better.

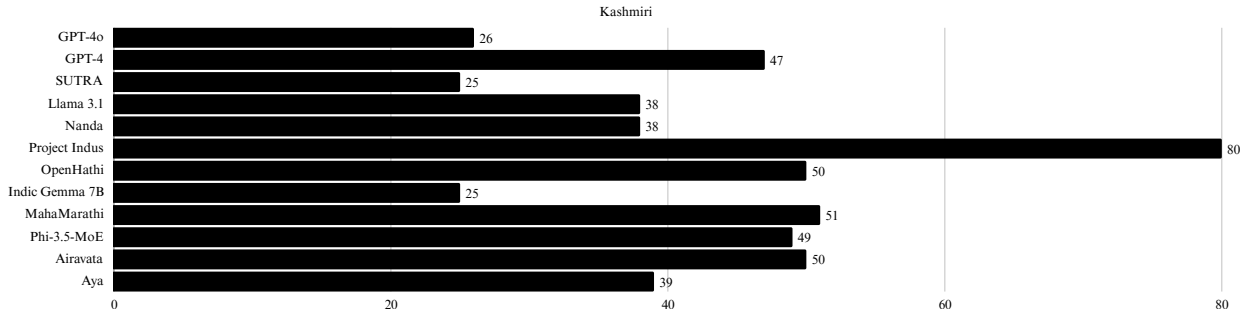

Figure 11: Number of tokens required for a single example text in Kashmiri. Lower values are better.

A.2 Example Texts Used for Tokenizer Evaluation

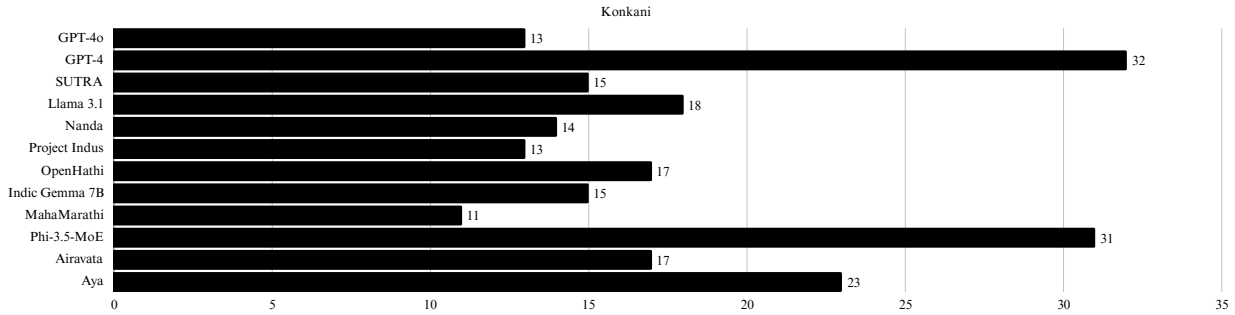

Figure 12: Number of tokens required for a single example text in Konkani. Lower values are better.

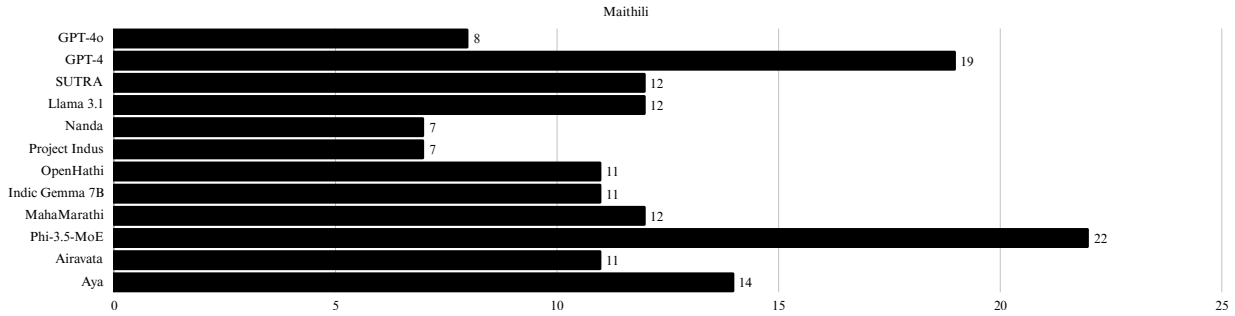

Figure 13: Number of tokens required for a single example text in Maithili. Lower values are better.

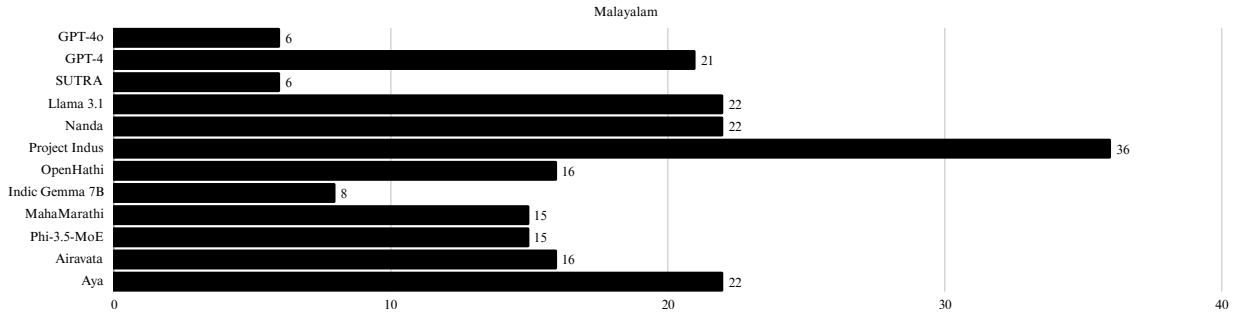

Figure 14: Number of tokens required for a single example text in Malayalam. Lower values are better.

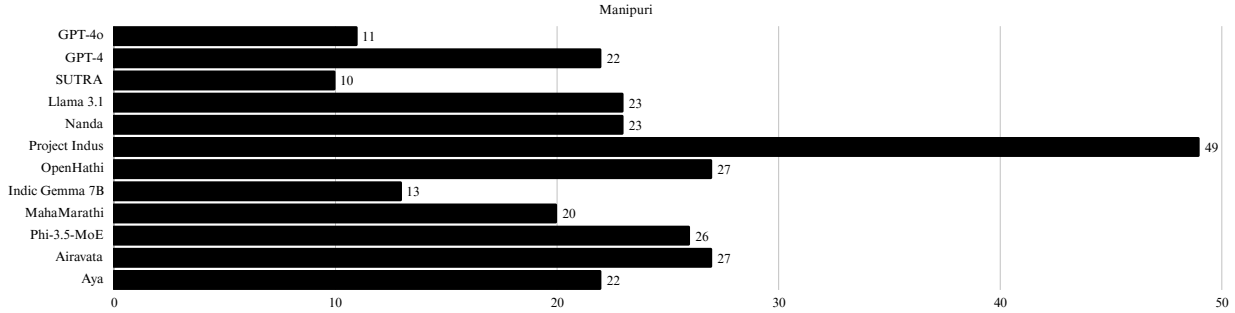

Figure 15: Number of tokens required for a single example text in Manipuri. Lower values are better.

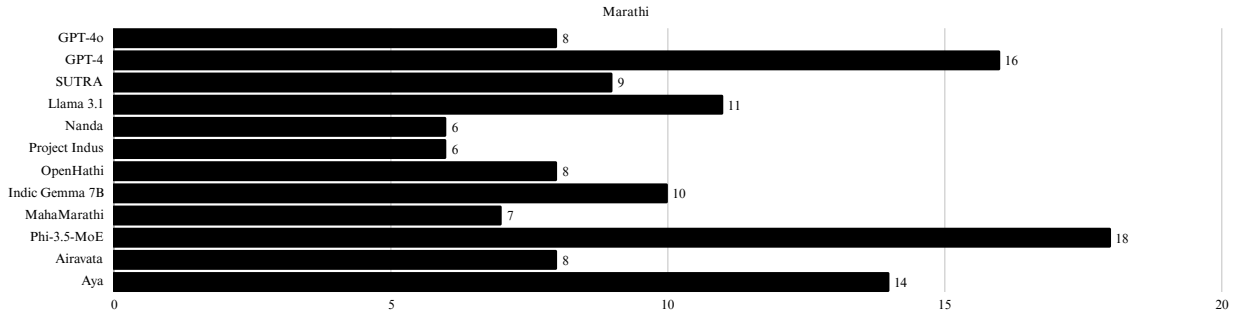

Figure 16: Number of tokens required for a single example text in Marathi. Lower values are better.

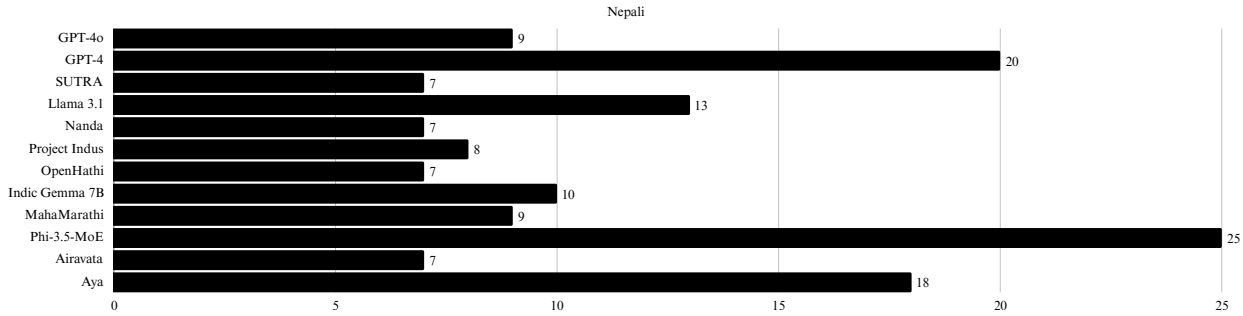

Figure 17: Number of tokens required for a single example text in Nepali. Lower values are better.

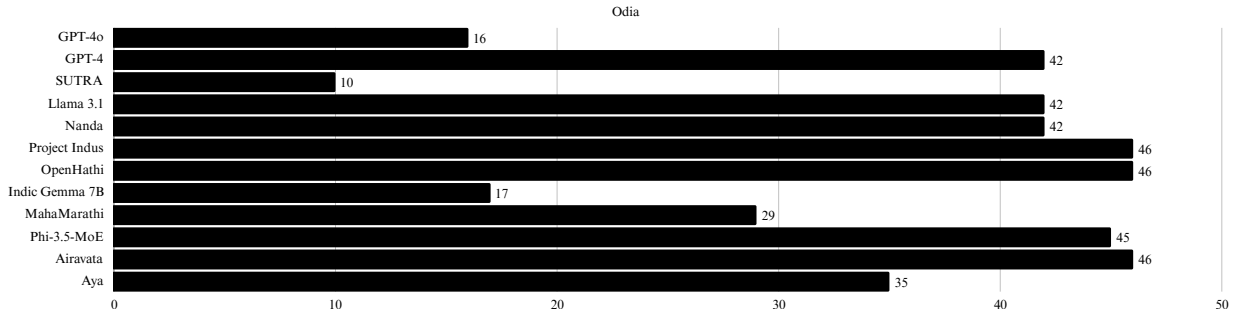

Figure 18: Number of tokens required for a single example text in Odia. Lower values are better.

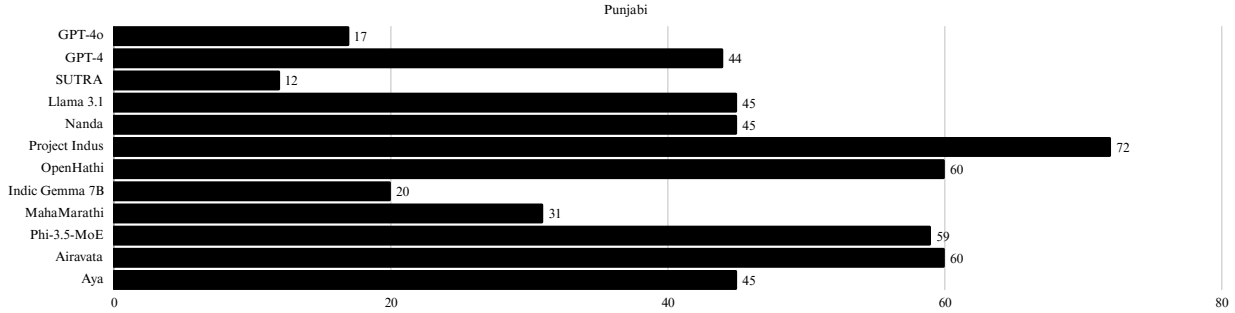

Figure 19: Number of tokens required for a single example text in Punjabi. Lower values are better.

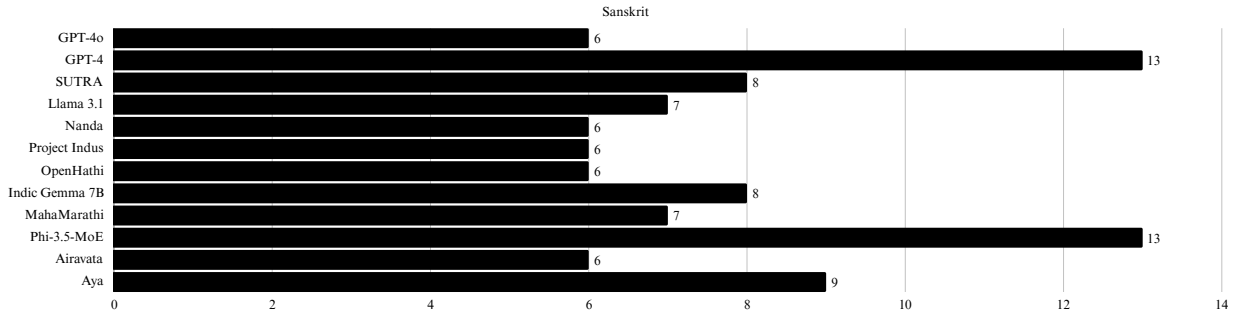

Figure 20: Number of tokens required for a single example text in Sanskrit. Lower values are better.

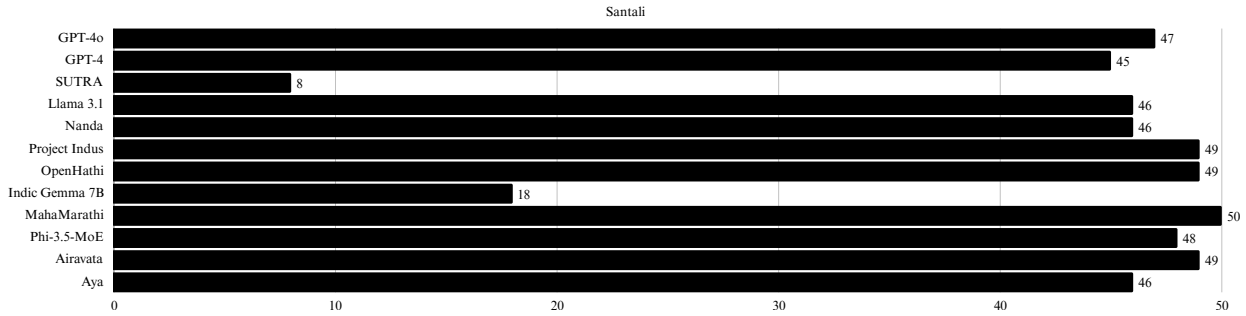

Figure 21: Number of tokens required for a single example text in Santali. Lower values are better.

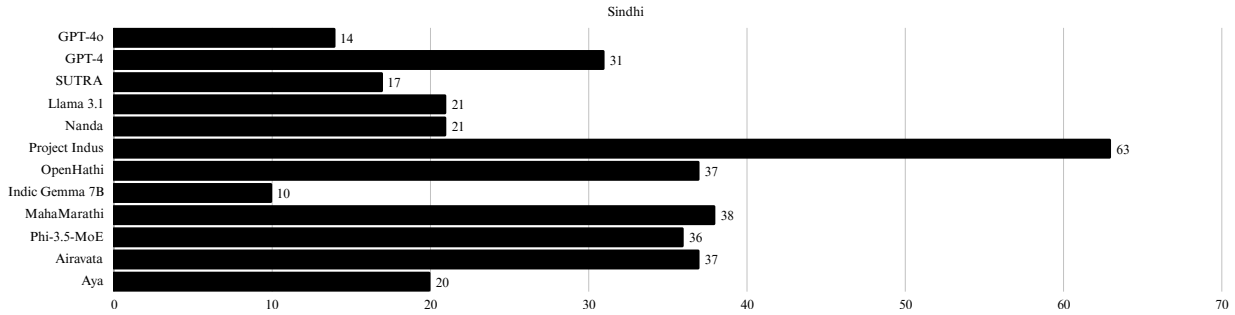

Figure 22: Number of tokens required for a single example text in Sindhi. Lower values are better.

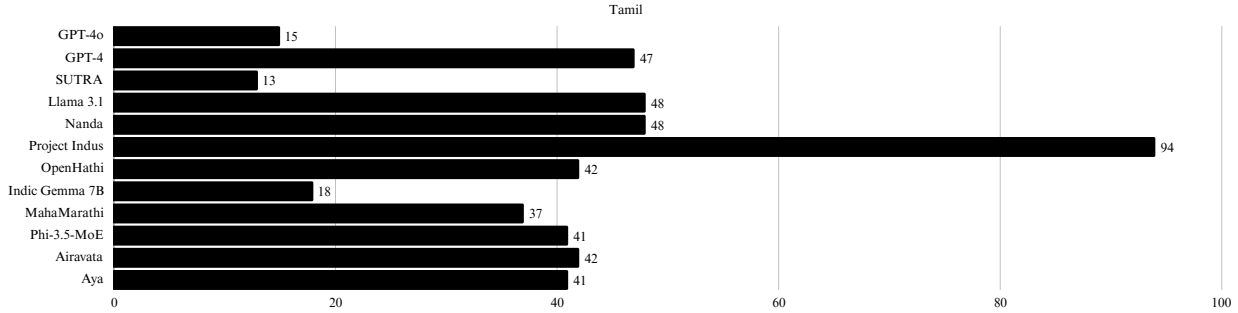

Figure 23: Number of tokens required for a single example text in Tamil. Lower values are better.

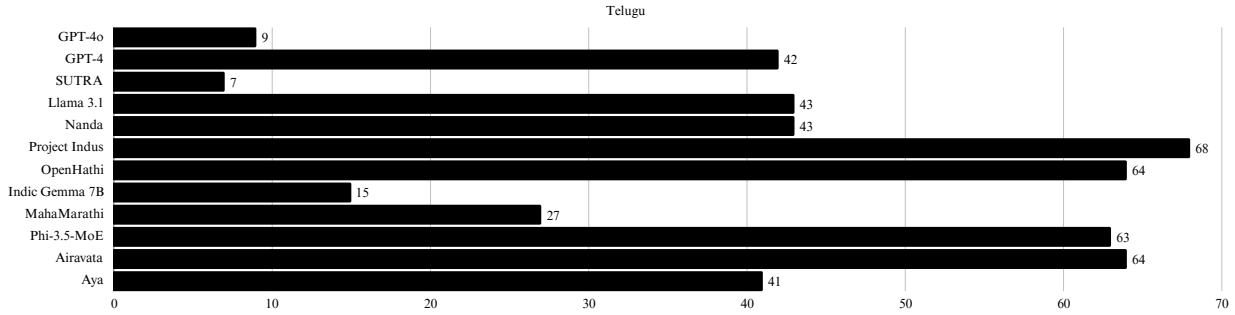

Figure 24: Number of tokens required for a single example text in Telugu. Lower values are better.

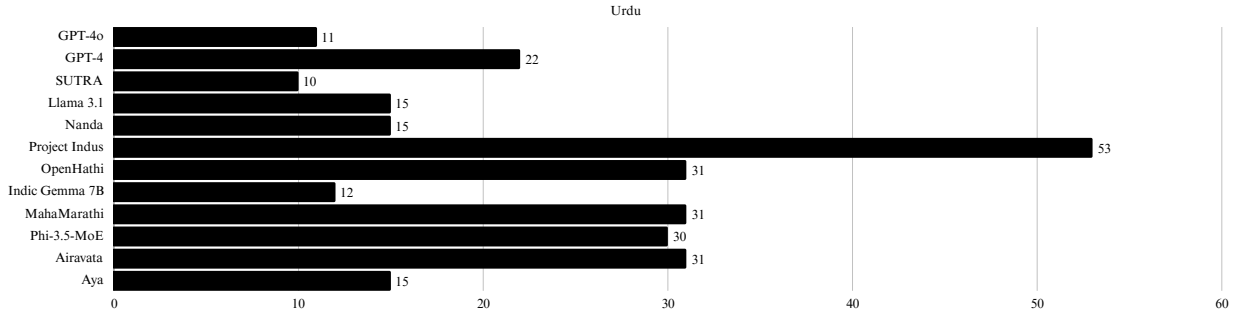

Figure 25: Number of tokens required for a single example text in Urdu. Lower values are better.

Example Text in Assamese

"জীৱনৰ পৰিসৰে মোহিত হোৱাটো বাঞ্ছনীয়।"
Translation: "It is desirable to be captivated by the expanse of life."
Source: "Miri Jiyori" by Rajanikanta Bordoloi

Example Text in Bengali

"পৃথিবীর বিভিন্ন প্রান্তে মানুষের নানা সংস্কৃতি রয়েছে"
Translation: "There are various cultures of people across different parts of the world."
Source: General expression

Example Text in Bodo

"बोरो जाति हिन्सा नाकुं आनि एकोफ फयलाइ"
Translation: "The Bodo people believe in peace and unity."
Source: Cultural Belief

Example Text in Dogri

"डोगरी जुबां साडा मान साडा अभिमान"
Translation: "Dogri language, our pride and honor."
Source: Cultural Belief

Example Text in Gujarati

"જનની જનમભૂમિ સ્વર્ગથી પણ મહાન છે"
Translation: "Mother and motherland are greater than heaven."
Source: General Expression (A widely known patriotic sentiment)

Example Text in Hindi

"मधुर मधुर मेरे दीपक जल"
Translation: "Burn gently, my lamp."
Source: Mahadevi Verma, Madhur Madhur Mere Deepak Jal (1930)

Example Text in Kannada

"ಹರಿದ ಹವೆಯಂತೆ ನಿನ್ನ ಪ್ರೀತಿ"
Translation: "Your love, like the passing wind."
Source: Kuvempu, Sri Ramayana Darshanam (1949)

Example Text in Kashmiri

"بی چھ مہاراز روز بَون تہِ منزِ چھ شام"
Translation: "You are my king, you live in my evening and dawn."
Source: Cultural Belief (Reflects themes in Kashmiri Sufi poetry)

Example Text in Konkani

"जेवपाच्या ताटात एक चटणी खासच"
Translation: "A special condiment on the dining plate."
Source: Cultural Belief (Consistent with Konkani folklore themes)

Example Text in Maithili

"हमर मिथिला धरोहर छै"
Translation: "Our Mithila is a heritage."
Source: Cultural Belief (Represents pride in Mithila heritage)

Figure 26: Example Texts for Assamese, Bengali, Bodo, Dogri, Gujarati, Hindi, Kannada, Kashmiri, Konkani, Maithili.

Example Text in Malayalam

"വാർണം പൂക്കൾ"
Translation: "Colorful flowers."
Source: Changampuzha Krishna Pillai, Varnangal (1940)

Example Text in Manipuri

"ঈশ্বর শংদন চিত্রং"
Translation: "O Lord, give me strength."
Source: General Devotional Theme

Example Text in Marathi

"स्वप्नातली सोनपरी"
Translation: "The golden fairy of dreams."
Source: Marathi Poets and Folklore, Marathi Literature (Generalized representation)

Example Text in Nepali

"मेरा देशको माया माया छ"
Translation: "My love for my country is immense."
Source: Patriotism in Nepali Poetry

Example Text in Odia

"ବନ୍ଦେ ଉତ୍କଳ ଜନନୀ"
Translation: "I bow to Mother Utkala."
Source: Laxmibai, Odia Patriotism (Published in 1947)

Example Text in Punjabi

"ਸੰਤੋਖੀ ਰਖੀਏ ਰੱਬ ਨਾਲ ਮਿੱਤਰੋ"
Translation: "Be content, my friends, with God."
Source: Punjabi Proverbs and Spirituality, Punjabi Folklore (Generalized spirituality)

Example Text in Sanskrit

"सत्यमेव जयते"
Translation: "Truth alone triumphs."
Source: Mundaka Upanishad, Shruti Texts of Hinduism (1500 BCE)

Example Text in Santali

"ᱚᱱᱞᱚᱞ ᱚᱞᱚᱞᱚᱭᱟᱜ ᱫᱟᱭᱚᱞᱟᱜᱟᱱ"
Translation: "The moon shines brightly in the sky."
Source: General Folk Expression

Example Text in Sindhi

"محبت م سچائي، دل کي سکون ملندو آهي"
Translation: "In love, truth brings peace to the heart."
Source: Sindhi Poetry, Sufism and Love (1982)

Example Text in Tamil

"காற்றின் இசை காதில் மழைத் துளி போல"
Translation: "The sound of the wind, like a raindrop in the ear."
Source: Subramania Bharati, Tamil Poems and Songs

Example Text in Telugu

"ప్రతీ రాత్రి తరువాత ఉదయం"
Translation: "After every night, comes the morning."
Source: General Expression (Hope themes in Telugu literature)

Example Text in Urdu

"چاندنی رات میں، دریا کا سکوت"
Translation: "The silence of the river on a moonlit night."
Source: Faiz Ahmed Faiz, Whispers of the Night

Figure 27: Example Texts for Maithili, Malayalam, Manipuri, Marathi, Nepali, Odia, Punjabi, Sanskrit, Santali, Sindhi, Tamil, Telugu, Urdu.